Exam for the course DAT171 Object oriented programming in Python

Time: 19th March 2019 8:30-13:30

Teacher: Thomas Svedberg (031-772 1522), Mikael Öhman (073-6837674)

Permitted aids: Cay Horstmann: Python for everyone. Manuals and lecture notes are available on the computers.

Teacher will visit the rooms: Around 10:00 and 12:00

Formalities: In each file you hand in, you should write your examination code. Also enter the computer number (from the exam cover).

The documentation and lectures notes are available on `C:\__EXAM__`. Verify that these files are there immediately. Handing in the code should be done under the same folder: `C:\__EXAM__\Assignments\`, when you are finished. If you do not store the files here, they are not included in the exam. Save the files for each question in the appropriate folder, e.g. `C:\__EXAM__\Assignments\Question1\`. *Make sure you only hand in one solution for each question, otherwise you will receive **zero** points.* Complex sections of your code should have a descriptive comment of what is achieved.

When you finish the exam you should log out and fill in the (empty) exam cover page like normal and hand this in at the end of the exam.

Corrections: The results will be announced through Ladok, on or before April 2nd. The review will be the same day (April 2nd) 12:20-13:00 at Hörsalsvägen 7b, 3:rd floor.

Grading: There is a total of 25 points which yields grades at the standard 40%, 60%, and 80% limits:

```python
def grade(points):
    if points >= 20:
        return 'Grade 5'
    elif points >= 15:
        return 'Grade 4'
    elif points >= 10:
        return 'Grade 3'
    else:
        return 'Fail'
```

Question 1: INI files (7p)

INI-files are commonly used for program configurations. They use simple text based formats with support for sections.

1. Empty lines as well as lines with only whitespace are ignored.
2. Lines with '#' as the first non-whitespace character are commented (ignored)
3. Any line starting with '[' denotes a section, e.g. `[Graphics]`
4. Parameters and values are stored as `Parameter=Value`

Open `default_config.ini` in PyCharm to see what a typical file looks like.

While there are specialised libraries for reading and writing this type of file, we want you to write the code yourself for this assignment.

Part A (2.5p)

Write the function `read_ini` that takes a filename and returns the configuration as a dictionary:

```
>>> conf = read_ini('default_config.ini')
>>> print(conf)
{'GamePlay': {'difficulty': 'medium', 'lives': '9', 'aim_assist': 'off'},
'Graphics': {'resolution': '1920x1080', 'texture_quality': 'high', 'geometry_detail': 'medium'},
'Sound': {'music': 'off', 'effects': 'on', 'volume': '100'}}
```

Raise a `ValueError` if an option is specified before any category.

Part B (2p)

Write the function `write_ini` that takes a filename and a dictionary (like the one you read in section one) and writes an ini file. There should be one empty line between sections.

```
>>> game_options = {'Graphics': {'resolution': '1920x1080', 'texture_quality': 'high'},
                    'Gameplay': {'difficulty': 'medium', 'lives': '9'}}
>>> write_ini('game_options.ini', game_options)
```

should result in a valid ini file, like:

```
[Graphics]
resolution=1920x1080
texture_quality=high

[Gameplay]
difficulty=medium
lives=9
```

Part C (2.5p)

Write a function `apply_overrides(default_config, user_overrides)` that takes a default configuration dictionary and a dictionary with user overrides.

The purpose of this function is to apply the user overrides on top of the default config.

It must print warnings when a user have specified an option or section that doesn't exist in the default config. In the case that a user has put a parameter in the wrong section, it should check it is listed in any of the other section, and give that as a hint.

```
>>> config = read_ini('default_config.ini')
>>> user = read_ini('user_overrides.ini')
>>> apply_overrides(config, user)
Section 'Cheats' not found in default config. Skipping.
Parameter 'volume' not found under section 'GamePlay'. Perhaps you meant to put it under 'Sound'?
Parameter 'filter' not found under section 'Graphics'.
>>> write_ini('config.ini', config)
```

Question 2: Image edge detection (5p)

A common task when working with images is edge detection, i.e. to find the edges of an area of a picture.

In this task we will explore the problems one find when doing edge detection. The example picture "animal.jpg" is the first of many images that contains a picture of an animal, we want to use these images as a base for a line sketch of the animals, and therefor want to find the edges automatically.

Part A (1p)

Read the image and plot it in in the top left area of a 2x2 subplot window.

If you look at the resulting variable after reading the file it should have the shape (NX, NX, 3). Here the last dimension are the Red, Green and Blue values for the pixel.

We want to have the gray-scale value here. To do so calculate the mean value of the RGB-numbers using a suitable method from NumPy.

Use this mean value image/array in the following tasks.

Hint: Look at `imageio`

Hint: To make the plot gray, use the `gray` *colormap from the Matplotlib module* `cm`

Part B (1p)

In `numpy.ndimage` there are many methods for manipulating image data. Using the `sobel` method calculate the filters for x- and y-directions of the image. Add the resulting images/filters to the plot (in subplots 2 and 3)

Part C (1p)

To get the final filter we need to take the geometric mean of the x- and y- filters ($sqrt(x * x + y * y)$), calculate this filter and using a suitable method from NumPy and add it in the final subplot.

Hint: While the xy-filters have dtype uint8, the results from NumPy might have a dtype that is not compatible with the method you use to show the image. If this is the case, try using the filter.astype() method when showing the image.

Part D (2p)

As can be seen from the images from above, the result can not be used for anything at this point! This is due to the noise in the picture.

To reduce the noise, find a suitable method in `ndimage` to perform Gaussian smoothing on the image before calculating the filter. Re-generate the plots for a few smoothing parameters, and finally choose the one that you think produce a reasonable result.

Question 3: Smart OuterMatrix (6p total)

Outer products appear frequently in applications dealing with linear algebra. An outer product of 2 vectors, u and v, result in a matrix, with coefficients $M_{ij} = u_i v_j$

While one can construct such a matrix explicitly, such a block of numbers would take up much more memory, and many operations can be done much more efficiently:

1. Dot product, with vector x:
$$M \cdot x = \sum_j M_{ij} x_j = u_i v_j x_j = u(v \cdot x)$$
By doing the dot product first, we need only do $O(N)$ operations, as opposed to $O(N^2)$.
2. Scaling, with scalar a (you simply scale either vector):
$$M\ a = M_{ij} a = u_i (v_j a)$$
3. Matrix norm (the Frobenius norm):
$$|M| = \sqrt{\sum_{ij} M_{ij}^2} = \sqrt{\sum_i u_i^2} \sqrt{\sum_j v_j^2} = |u|\ |v|$$

By not expanding the matrix directly, we can thus perform much more efficient Matrix operations!

Part A (0.5p)

Write a class `OuterMatrix` that takes 2 NumPy arrays as inputs, and stores *copies* of them internally. This class will represent a matrix, without explicitly computing the outer product.

Part B (0.5p)

Just like other types of special matrices (like sparse matrices), implement the method `toarray()` that computes an expanded NumPy array. *Hint* ***`numpy.outer`***

Part C (0.5p)

Transposing is also common. Add the method `transpose()` which returns a transposed copy of type `OuterMatrix`.

Part D (0.5p)

Implement the efficient (Frobenius) norm as the method `norm`. *Hint* ***`numpy.linalg.norm`***

Part E (1.5p)

To make this matrix actually behave like a matrix, you must now implement indexing for *getting* values off this matrix. You should support slices and integer values (see the 4 cases in the example below)

Part F (1.5p)

Implement efficient multiplication for these cases:

```
x = np.array( ... )
m = OuterMatrix(u, v)
y = m * x  # performs dot product
m2 = m * 3.14  # computes a scaled matrix
m *= 2  # scales m
```

If the type isn't a scalar (`int`, `float`) or a 1D NumPy array; raise a *suitable* error.

Test

```
import numpy as np
a = np.array([9., 1., 2., 3., 4., 5., 6.])
b = np.array([2., 1., 4., 3., 6., 5., 7.])
c = np.array([1., 2., 1., 7., 8., 2., 4.])

m = OuterMatrix(a, b)

print("Expanded:\n{}".format(m.toarray()))
m_sub = m[1:3, 2:5]  # This should also be a OuterMatrix.
print("Slicing (expanded):\n{}".format(m_sub.toarray()))
print("Slicing:", m[1:3, 5])  # slice + int -> ndarray
print("Slicing:", m[3, 2:5])  # int + slice -> ndarray
print("Item:", m[2, 4])  # int + int -> float
print("Norm:", m.norm())
m *= 2.0
print("Norm after scaling:", m.norm())
print("Multiplication with array:", m * c)
m_sub_t = m_sub.transpose()
print("Transpose (expanded):\n{}".format(m_sub_t.toarray()))
# Exceptions
#print(m * c.tolist())
#print(m * np.stack((c ,c)))
```

Output

```
Expanded:
[[18.  9. 36. 27. 54. 45. 63.]
 [ 2.  1.  4.  3.  6.  5.  7.]
 [ 4.  2.  8.  6. 12. 10. 14.]
 [ 6.  3. 12.  9. 18. 15. 21.]
 [ 8.  4. 16. 12. 24. 20. 28.]
 [10.  5. 20. 15. 30. 25. 35.]
 [12.  6. 24. 18. 36. 30. 42.]]
Slicing (expanded):
[[ 4.  3.  6.]
 [ 8.  6. 12.]]
Slicing: [ 5. 10.]
Slicing: [12.  9. 18.]
Item: 12.0
Norm: 155.1773179301666
Norm after scaling: 310.3546358603332
Multiplication with array: [2070.  230.  460.  690.  920. 1150. 1380.]
Transpose (expanded):
[[ 4.  8.]
 [ 3.  6.]
 [ 6. 12.]]
```

Question 4: Flexible log sinks (7p total)

Larger complex software needs to customizable logging. A typical approach is to construct a "sink" that can receive messages.

Part A (1p)

Make a (comparable) enum called `LogLevel` with increasing severity: `debug`, `info`, `warning`, `error`.

Part B (2p)

Write a base class `LogSink` that takes parameter `minimum_log_level`. It should have

- an abstract method `_write(self, entry: str)`
- a static method `datestamp() -> str` that returns the current date and time in ISO format; e.g. `'[2019-03-19T11:31:23.135252]'`
- a method `push(self, importance: LogLevel, msg: str)` that, if the importance level is at or above the minimum log level, prepend the datestamp to the message and send the resulting string to `_write`.

Hints: *`datetime.datetime.now`*

Part C (0.5p):

Implement the subclass `PrintSink` that simply prints the output.

Part D (0.5p):

Implement the subclass `FileSink` which in addition to the minimum log level also takes a filename of the file to write output to (one entry per line) *For the sake of managing the complexity of this question on the exam, you are allowed to "leave" this file handle open*

Part E (1p):

Implement the subclass `MemorySink` which in addition to the minimum log level also takes a `maxlen` parameter.

The 'MemorySink' class should store at a maximum 'maxlen' messages, when the maximum number of messages are logged, the oldest message should be overwritten.

Implement a suitable method which enables us to loop over the messages, see the example below.

Hint: check out *`collections.deque`*

Part F (2p)

Write a subclass `MultiSink` that takes a list of other `LogSinks`. Overload `push` so that we can put the exact same datestamp on all the sinks.

Test

```
ps = PrintSink(LogLevel.info)
fs = FileSink(LogLevel.warning, 'logfile.txt')
multi = MultiSink([ps, fs])

multi.push(LogLevel.info, 'Multi-test: Stuff happened.')
multi.push(LogLevel.error, 'Multi-test: An error occurred!')

ms = MemorySink(LogLevel.warning, 5)
for test_message in ['A', 'B', 'C', 'D', 'E', 'F', 'G']:
    ms.push(LogLevel.warning, test_message)

for event in ms: # Should display C trough G
    print('MemorySink:', event)
```

Output

```
[2019-03-12T17:51:34.532546] - Multi-test: Stuff happened.
[2019-03-12T17:51:34.532585] - Multi-test: An error occurred!
MemorySink: [2019-03-12T17:51:34.532621] - C
MemorySink: [2019-03-12T17:51:34.532623] - D
MemorySink: [2019-03-12T17:51:34.532625] - E
MemorySink: [2019-03-12T17:51:34.532626] - F
MemorySink: [2019-03-12T17:51:34.532628] - G
```

Logfile should contain

```
[2019-03-12T17:51:34.532585] - Multi-test: An error occurred!
```